Size Vs Coherence Documentation

The experiment should run without anything being changed. Only one change should be made. These are ScreenWidth & ScreenHeight variables in Setup.m, which should be corrected to the screen being used.

Contents

Setup.m
Trial_Parameters.m
Options.m
Instructions.m
Main.m
Main_Loop.m
Output.m

Functions

VisualDegreesToPixels.m
RDK_Motion.m

Setup.m

One thing which always needs to be manually changed: ScreenWidth & ScreenHeight & ViewingDistance. These need to be adjusted to the cm of the screen being used. Otherwise target locations will be wrong.

This script creates the setup for the experiment. This includes things such as;

- Adding relevant output, input, and function folders to the path allowing them to be called whenever
- Setting up psychtoolbox and relevant screen information
- Defining screen information
- Calculating pixels per degree
- Setting desired frame rate
- Fixation cross parameters

Trial_Parameters.m

This script defines all variables which are used within the rest of the experiment and Main_Loop.m.

Below is a list of important variables you can search for and are changeable:

- **RDK_Radius_Sizes** - this defines the pixel sizes of the radii (can set to any realistic number). Further, the amount of sizes you give is the levels of this variable
- **coherencePercentageArray** - this defines the coherence percentages (0-1)
- **Repetitions** - number of repetitions of each size or coherence
- **Locations** - input for locations in visual degrees (x, y)
- **maxTime** - time in seconds each target is presented
- **dotSize** - size of each individual dot in px
- **dotColour** - colour of dots (0-1)
- **dotVelocity** - speed of dots (px per frame)
- **dotTimers** - time before each dot is reset (random dots only)

Options.m

This script is used to turn on and off some optional parameters. A short description of each can be seen below.

targetVisualise - draws an outline of the circumference of the current RDK target.
RDK_Fixation - draws a red dot within the center of the current RDK target
NOMOVEMENTNOISE - turns off all dot movements outside of the current RDK target
alphaMask - applies a mask based on some inputted sensitivity matrix. This mask changes the colour of dots to be closer to the background screen colour in less sensitive areas in attempt of making a ‘glaucoma like’ effect (***not implemented in this experiment***)
FPS_Show - this turns on a fps counter in the top left (usually used for debugging)

Instructions.m

This script is just simply where you type in the instructions you want displayed on screen. The design can also be customised from this script.

Main.m

Main.m is a script which just calls all other necessary scripts from a central place. It also has some variables and parameters you can name and set. **This is the script you want to run to run the experiment.**

- **participant_ID** - Just allows a string input
- **kbWait(0, 2)** - waits for a keyboard response and then continues after a key is pressed (used to continue from instructions)
- **WaitSecs(1)** - these are just some timers being used to break up the start of the experiment
- **averageFPS** - calculated the averageFPS which the experiment is running at
- **averageDensity** - calculated the average density at each quadrant of screen (TL is top left etc)

Main_Loop.m

This script runs the main experimental loop which is in charge of the whole experiment, from motion, to trial number etc. Everything is set within a while loop which increments the trial number after a response is given. The while loop is necessary due to the dynamic nature of the experiment.

Output.m

This script is quite messy at the moment.

This script creates all the output from the experiment, saving them to tables, csv's, etc. It also creates some plots and stores these in the 'Output Files' folder.

Tables:

- **finishedTable** - this table stores all the information based on main responses to variables. Including the correct direction of motion, the answer in key which was given, the accuracy (0 wrong, 1 right), coherence amount on the trial, radius size of target on the trial, and the response time to the target
- **thresholdTable** - this table combines accuracy, coherence, and size, and response time both (raw and log) into a matrix which is easier to manipulate into plots.
- **summaryTable** - this table uses the thresholdTable and computes the average response time, and accuracy ration, for each unique combination of coherence and size.
- **groupedDataCoherence & Size** - these tables just store and calculate the mean response time (raw and log) for either the Coherence, or Size data exclusively

Plots:

- **CoherenceSize_PsychFunc** - this plots the psychometric functions of coherence and size based on accuracy ratio. It separates each other level apart, so there are as many lines as the corresponding variable making one line for each size at every coherence level. It plots them in one figure but as subplots.
- **RTCombinedFig** - this plots the average response time over group size and coherence into two separate subplots
- **LogRTCombinedFig** - this is the exact same as above plot but instead uses log response time
- **heatmapSvC** - this plot creates a heat map based on the grouped instances of each matched size and coherence. The data is the average response time of all repetitions at a given size and coherence. So each square represents the response time of lets say 10% coherence at 100px size. This plot aims to display the trend of performance between size and coherence to see which is more impactful on performance.
- **DRP** - this stands for 'Dynamic Radius Plot'. It essentially plots all targets in the location they were presented and at the radius given. It also changes colour of

the circumferences based on either response time or radius. It is not so relevant for this experiment and is more used for debugging.

- **Circle2.m** - this is just a small function which is used to create and draw circles to plot in the DRP
- **Saveas** - all plots are saved using the saveas function which saves them into the ‘Output Files’ folder with the relevant name of the plot and the participant ID.

VisualDegreesToPixels.m

This function is used to convert inputted visual degrees into on screen pixel coordinates. It uses the pixels per degree of screen to calculate this.

convertedAngles = VisualDegreesToPixels(degreeLocations, width, height, ppd)

- **convertedAngles** - returns the converted pixel coords
- **degreeLocations** - array input of defined visual angles
- **Width** - screen horizontal resolution in pixels
- **Height** - screen vertical resolution in pixels
- **Ppd** - this is defined in *Setup.m*. But, to briefly define, it is the pixels per degree of screen calculated from the fov of the screen divided by the horizontal resolution (or largest resolution)

RDK_Motion.m

```
function [movementPos, xRandomValues, yRandomValues, newPath, dotRadians] =
RDK_Motion(noDots, Speed, width, height, ...
   coherence, movementPos, insideRadius, RDK_Radius, RDK_Center, RDK_Direction,
Index, NOMOVEMENTNOISE, ...
   xRandomValues, yRandomValues, newPath, dotRadians)
```

[movementPos, xRandomValues, yRandomValues, newPath, dotRadians] - Returned variables are necessary to update the logic in the Main_Loop script.

RDK_Motion(noDots, Speed, width, height, coherence, movementPos, insideRadius, RDK_Radius, RDK_Center, RDK_Direction, Index, NOMOVEMENTNOISE, xRandomValues, yRandomValues, timePassed, newPath, dotRadians)

- **noDots** - takes the amount of dots of experiment to use as the for loop length
- **Speed** - defines the speed at which all dots move (this is in pixels per frame, so a speed of 1 is 1 pixel per frame)
- **InsideRadius** - this variable is a logical array which tracks if dots are withinside the current RDK target radius
- **RDK_Radius** - current radius size of RDK target
- **RDK_Center** - defines the center of the current RDK target (in x, y coords)
- **RDK_Direction** - an array of 4 integers [-1, 1, -1, 1]. Used to define coherent RDK motion direction.
- **Index** - gives a random index for the aforementioned RDK Direction (if > 2 the movement is defined as up or down)
- **NOMOVEMENTNOISE** - this is an optional parameter (see Options.m) which turns of all motion outside of the current RDK target
- **xRandomValues & yRandomValues** - these are arrays of the cos() and sin() of randomly generated radians. It allows for 360 movements in random dots.
- **newPath** - this is a logical array which keeps track of random dots outside of RDK target, and indicates whether they need to be given a new direction, after a randomly set timer has passed.
- **dotRadians** - these are randomly generated radians used to define 360 degree movements for dots